

```
import tensorflow as tf

from sklearn import metrics
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

from tqdm.notebook import tqdm
import random
import warnings

warnings.filterwarnings("ignore")
```

```
(trainX, trainY), (testX, testY) = tf.keras.datasets.fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 2s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 1s 0us/step
```

```
trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
testX = testX.reshape((testX.shape[0], 28, 28, 1))
```

```
trainY_cat = tf.keras.utils.to_categorical(trainY)
testY_cat = tf.keras.utils.to_categorical(testY)
```

```
train_norm = trainX.astype('float32')
test_norm = testX.astype('float32')
```

```
train_norm = train_norm / 255.0
test_norm = test_norm / 255.0
```

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress',
               'Coat', 'Sandal', 'Shirt', 'Sneaker',
               'Bag', 'Ankle boot']
```

```
plt.figure(figsize=(10,10))

for i in range(25):
    plt.subplot(5, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(trainX[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[trainY[i]])

plt.show()
```



```
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, kernel_size=(3, 3),
        input_shape=(28, 28, 1),
        activation='relu'),
    tf.keras.layers.AveragePooling2D(pool_size=(2, 2),
        name='pooling_layer_1'),
    tf.keras.layers.Conv2D(64, kernel_size=(3, 3),
        activation='relu',
        padding='same'),
    tf.keras.layers.AveragePooling2D(pool_size=(2, 2),
        name='pooling_layer_2'),
    tf.keras.layers.GlobalAveragePooling2D(name='pooling_layer_3'),
    tf.keras.layers.Dense(len(class_names),
        activation='softmax',
        name='output_layer')
])
```

```
model.compile(loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy'])
```

```
history = model.fit(train_norm, trainY_cat,
    epochs=5,
    validation_data=(test_norm, testY_cat))
```

```
Epoch 1/5
1875/1875 — 64s 33ms/step - accuracy: 0.6110 - loss: 1.1043 - val_accuracy: 0.7085 - val_loss: 0.8506
Epoch 2/5
1875/1875 — 60s 32ms/step - accuracy: 0.7243 - loss: 0.7716 - val_accuracy: 0.7454 - val_loss: 0.7316
Epoch 3/5
1875/1875 — 61s 32ms/step - accuracy: 0.7541 - loss: 0.6926 - val_accuracy: 0.7487 - val_loss: 0.6953
```

Epoch 4/5

1875/1875  **60s** 32ms/step - accuracy: 0.7721 - loss: 0.6441 - val_accuracy: 0.7794 - val_loss: 0.6290

Epoch 5/5

1875/1875  **81s** 32ms/step - accuracy: 0.7854 - loss: 0.6056 - val_accuracy: 0.7889 - val_loss: 0.6177

```
tf.keras.utils.plot_model(model, show_shapes=True)
```


Conv2D

Input shape: (None, 28, 28, 1)

Output shape: (None, 26, 26, 32)

```
model.summary()
```

Model: "sequential"

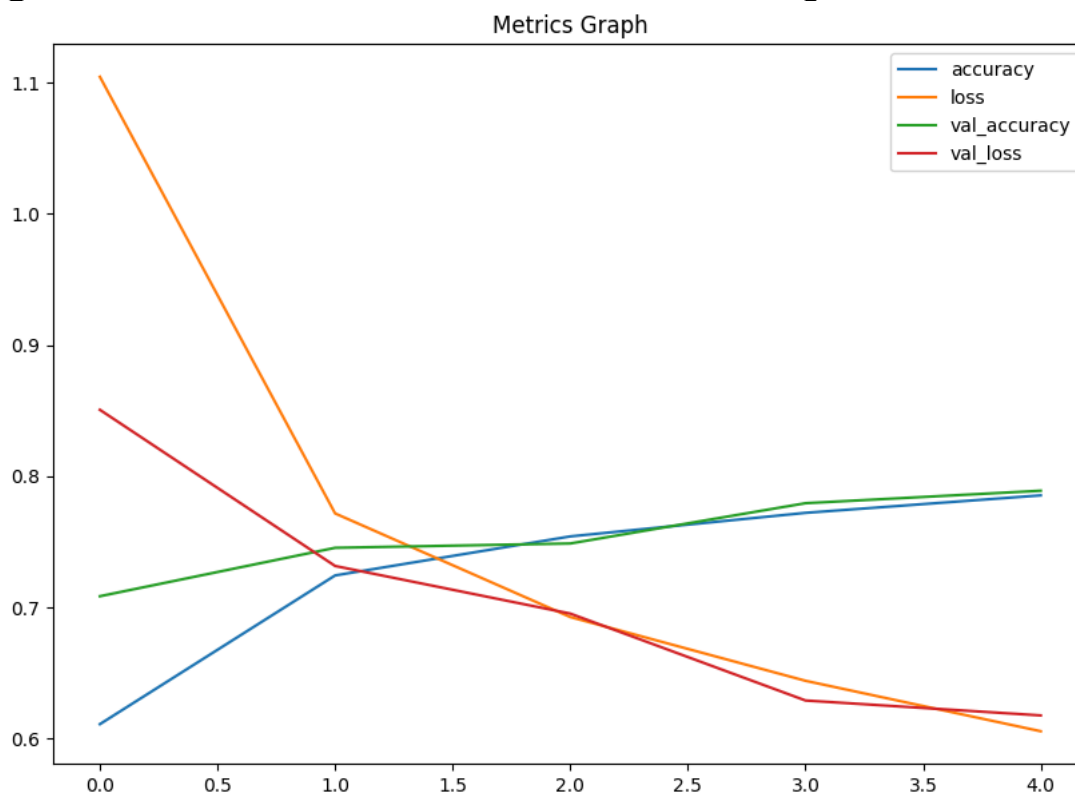
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
pooling_layer_1 (AveragePooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 12, 12, 64)	18,496
pooling_layer_2 (AveragePooling2D)	(None, 6, 6, 64)	0
pooling_layer_3 (GlobalAveragePooling2D)	(None, 64)	0
output_layer (Dense)	(None, 10)	650

Total params: 58,400 (228.13 KB)
 Trainable params: 19,466 (76.04 KB)
 Non-trainable params: 0 (0.00 KB)

Conv2D

```
pd.DataFrame(history.history).plot(figsize=(10, 7))
```

```
plt.title("Metrics Graph")  
plt.show()
```



```
model.evaluate(test_norm, testY_cat)
```

```
313/313 ————— 4s 12ms/step - accuracy: 0.7889 - loss: 0.6177
[0.617668092250824, 0.7889000177383423]
```

```
predictions = model.predict(test_norm)
```

```
313/313 ————— 4s 13ms/step
```

```
predicted_labels = tf.argmax(predictions, axis=1)
```

```
y_test = tf.argmax(testY_cat, axis=1)
```

```
y_test = tf.Variable(y_test)
```

```
print(metrics.accuracy_score(y_test, predicted_labels))
```

```
0.7889
```

```
print(metrics.classification_report(y_test, predicted_labels))
```

	precision	recall	f1-score	support
0	0.84	0.64	0.73	1000
1	0.97	0.93	0.95	1000
2	0.79	0.57	0.66	1000
3	0.70	0.86	0.77	1000
4	0.60	0.71	0.65	1000
5	0.92	0.92	0.92	1000
6	0.45	0.52	0.48	1000
7	0.86	0.93	0.89	1000
8	0.94	0.94	0.94	1000
9	0.95	0.87	0.91	1000
accuracy			0.79	10000
macro avg	0.80	0.79	0.79	10000
weighted avg	0.80	0.79	0.79	10000

```
cm = metrics.confusion_matrix(y_test, predicted_labels)
```

```
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
```

```
plt.show()
```